

Python PhoneBook Backend Knowledge Bytes

<https://gemini.google.com/app/8e79e6d1ae22b0a1>

User prompt: You are acting as a patient senior engineer walking a teammate through an unfamiliar piece of code. Your job is to break the following code document into a series of small, self-contained explanations called "KNOWLEDGE BYTES." RULES FOR KNOWLEDGE BYTES: 1. Each byte covers ONE concept, function, block, or responsibility — never more. 2. Each byte must be understandable on its own, but should note what earlier bytes it builds on (if any). 3. Order the bytes so understanding builds progressively: context/purpose first, then structure, then details, then edge cases/gotchas last. 4. Use plain language first, then show the relevant code snippet, then explain it. 5. Avoid restating the code line-by-line — explain WHAT it does and WHY it exists, not just a transliteration of syntax into English. 6. Keep each byte short enough to read in under 45 seconds. 7. Do not assume prior knowledge of this specific codebase — define any domain-specific terms or patterns the first time they appear. OUTPUT FORMAT (repeat for each byte): --- ### Byte [N]: [Short, descriptive title] **Builds on:** [Byte # or "None — starting point"] **In plain terms:** [1-3 sentences explaining the concept/purpose in everyday language] **The code:** [language] [relevant snippet only — not the whole file] from datetime import datetime, timedelta from fastapi import APIRouter, Depends, HTTPException, Query from sqlalchemy import exists, func, or_ from sqlalchemy.exc import IntegrityError from sqlalchemy.orm import Session from .. import models from ..database import get_db from ..schemas import (ContactCreate, ContactImportRequest, ContactImportResponse, ContactListResponse, ContactMetricsResponse, ContactResponse, ContactUpdate, TagResponse,) router = APIRouter(prefix="/contacts", tags=["Contacts"],) def get_or_create_tags(db: Session, names: list[str]): """Return existing tags and create only genuinely new tag names. Tag matching is case-insensitive so imports such as Work/work do not create two visually identical tags. """ cleaned_names = [] seen = set() for name in names: cleaned = str(name).strip() key = cleaned.lower() if not cleaned or key in seen: continue seen.add(key) cleaned_names.append(cleaned) if not cleaned_names: return [] existing = db.query(models.Tag).filter(func.lower(models.Tag.name).in_(name.lower() for name in cleaned_names))).all() existing_by_name = {tag.name.lower(): tag for tag in existing} tags = [] for name in cleaned_names: tag = existing_by_name.get(name.lower()) if tag is None: tag = models.Tag(name=name) db.add(tag) db.flush() existing_by_name[name.lower()] = tag tags.append(tag) return tags # CREATE @router.post("/", response_model=ContactResponse) def create_contact(contact: ContactCreate, db: Session = Depends(get_db),): new_contact = models.Contact(name=contact.name, phone_number=contact.phone_number, email=contact.email, address=contact.address,) new_contact.tags = get_or_create_tags(db, contact.tags) db.add(new_contact) try: db.commit() db.refresh(new_contact) except IntegrityError: db.rollback() raise HTTPException(status_code=400, detail="Phone number or email already exists",) return new_contact # READ ALL + SEARCH + PAGINATION @router.get("/", response_model=ContactListResponse) def get_contacts(search: str | None = None, favorite: bool | None = None, tag: str | None = None, unlabeled: bool = False, recent: bool = False, sort: str = Query("name_asc", pattern="^(name_asc|name_desc|recently_viewed|recently_added|recently_updated)\$",), page: int = Query(1, ge=1), limit: int = Query(10, ge=1, le=100), db: Session = Depends(get_db),): query = db.query(models.Contact) if search: search_pattern = f"%{search.strip()}%" query = query.filter((models.Contact.name.ilike(search_pattern)) | (models.Contact.phone_number.ilike(search_pattern)) | exists(models.contact_tags.join(models.Tag.__table___.select() .where((models.contact_tags.c.contact_id == models.Contact.id) & (models.contact_tags.c.tag_id == models.Tag.id) & models.Tag.name.ilike(search_pattern)))) if favorite is not None: query = query.filter(models.Contact.is_favorite == favorite) if tag: query = query.join(models.Contact.tags).filter(models.Tag.name.ilike(tag)) if unlabeled: query = query.outerjoin(models.Contact.tags).filter(models.Tag.id.is_(None)) if recent: query = query.filter(models.Contact.last_viewed_at.is_not(None)) if sort == "name_desc": query = query.order_by(models.Contact.name.desc()) elif sort == "recently_viewed": query = query.order_by(models.Contact.last_viewed_at.desc().nullslast(), models.Contact.name.asc(),) elif sort == "recently_added": query = query.order_by(models.Contact.created_at.desc()) elif sort == "recently_updated": query = query.order_by(models.Contact.updated_at.desc()) else: query = query.order_by(models.Contact.name.asc()) total = query.count() offset = (page - 1) * limit contacts = query.offset(offset).limit(limit).all() return { "contacts": contacts, "page": page, "limit": limit, "total": total, } @router.get("/metrics", response_model=ContactMetricsResponse) def get_contact_metrics(db: Session = Depends(get_db)): total = db.query(func.count(models.Contact.id)).scalar() or 0 favorites = (db.query(func.count(models.Contact.id)) .filter(models.Contact.is_favorite.is_(True)) .scalar() or 0) unlabeled = (db.query(func.count(models.Contact.id)) .outerjoin(models.Contact.tags) .filter(models.Tag.id.is_(None)) .scalar() or 0) recently_added = (db.query(func.count(models.Contact.id)) .filter(models.Contact.created_at >= datetime.utcnow() - timedelta(days=7)) .scalar() or 0) return { "total": total, "favorites": favorites, "recently_added": recently_added, "unlabeled": unlabeled, } @router.get("/tags", response_model=list[TagResponse]) def get_tags(db: Session = Depends(get_db)): return db.query(models.Tag).order_by(models.Tag.name.asc()).all() @router.post("/import", response_model=ContactImportResponse) def import_contacts(payload: ContactImportRequest, db: Session = Depends(get_db),): imported = 0 skipped = 0 errors = [] for row_number, contact in enumerate(payload.rows, start=2): try: with db.begin_nested(): new_contact = models.Contact(name=contact.name, phone_number=contact.phone_number, email=contact.email, address=contact.address,) new_contact.tags = get_or_create_tags(db, contact.tags) db.add(new_contact) db.flush() imported += 1 except IntegrityError: skipped += 1 errors.append({ "row": row_number, "reason": "Duplicate phone number or email", }) db.commit() return { "imported": imported, "skipped": skipped, "errors": errors, } # READ ONE @router.get("/{contact_id}", response_model=ContactResponse) def get_contact(contact_id: int, db: Session = Depends(get_db),): contact = (db.query(models.Contact) .filter(models.Contact.id == contact_id) .first()) if contact is None: raise HTTPException(status_code=404, detail="Contact not found",) return contact # UPDATE @router.put("/{contact_id}", response_model=ContactResponse) def update_contact(contact_id: int, contact_data: ContactUpdate, db: Session = Depends(get_db),): contact = (db.query(models.Contact) .filter(models.Contact.id == contact_id) .first()) if contact is None: raise HTTPException(status_code=404, detail="Contact not found",) update_data = contact_data.model_dump(exclude_unset=True) tag_names = update_data.pop("tags", None) for field, value in update_data.items(): setattr(contact, field, value) if tag_names is not None: contact.tags = get_or_create_tags(db, tag_names) try: db.commit() db.refresh(contact) except IntegrityError: db.rollback() raise HTTPException(status_code=400, detail="Phone number or email already exists",) return contact # DELETE @router.delete("/{contact_id}") def delete_contact(contact_id: int, db: Session = Depends(get_db),): contact = (db.query(models.Contact) .filter(models.Contact.id == contact_id) .first()) if contact is None: raise HTTPException(status_code=404, detail="Contact not found",) db.delete(contact) db.commit() return { "message": "Contact deleted successfully" } @router.patch("/{contact_id}/viewed", response_model=ContactResponse) def mark_contact_viewed(contact_id: int, db: Session = Depends(get_db),): contact = (db.query(models.Contact) .filter(models.Contact.id == contact_id) .first()) if contact is None: raise HTTPException(status_code=404, detail="Contact not found",) contact.last_viewed_at = datetime.utcnow() db.commit() db.refresh(contact) return contact

Response: Byte 1: Router Setup & API Routing

Builds on: None — starting point

In plain terms: This code defines the routing namespace for all contact operations in a web service. It tells the web framework to group all the endpoints in this file under the shared web path /contacts.

The code:

```
router = APIRouter(
    prefix="/contacts",
    tags=["Contacts"],
)
```

FastAPI uses an **APIRouter** to organize endpoints into modular files rather than placing every URL route into a single monolithic script.

By defining `prefix="/contacts"`, any path defined below (such as `@router.get("/")` or `@router.post("/")`) automatically prefixes its URL with `/contacts`. The `tags=["Contacts"]` argument organizes these endpoints together inside the automatically generated OpenAPI (Swagger) interactive documentation.

Byte 2: Case-Insensitive Tag Deduplication

Builds on: Byte 1

In plain terms: When assigning labels (tags) to a contact, people often write the same word with different casing or accidental whitespace (like "Work" vs " work "). This helper function ensures the system reuses existing tags and only creates database records for genuinely new, cleaned tags.

The code:

```
def get_or_create_tags(db: Session, names: list[str]):
    cleaned_names = []
    seen = set()

    for name in names:
        cleaned = str(name).strip()
        key = cleaned.lower()

        if not cleaned or key in seen:
            continue

        seen.add(key)
        cleaned_names.append(cleaned)

    if not cleaned_names:
        return []

    existing = db.query(models.Tag).filter(
        func.lower(models.Tag.name).in_([name.lower() for name in cleaned_names])
    ).all()
    existing_by_name = {tag.name.lower(): tag for tag in existing}

    tags = []

    for name in cleaned_names:
        tag = existing_by_name.get(name.lower())

        if tag is None:
            tag = models.Tag(name=name)
            db.add(tag)
            db.flush()
            existing_by_name[name.lower()] = tag

        tags.append(tag)

    return tags
```

A **Session** (db) represents an active conversation with the database using **SQLAlchemy** (an Object-Relational Mapper, or ORM, which translates Python objects to database rows).

This function cleans the input list in memory, fetches any matching tags from the database in a single query using case-insensitive comparison (`func.lower`), and adds only missing tags. It uses `db.flush()` on newly created tags to immediately assign them database IDs without finalizing the entire database transaction prematurely.

Byte 3: Listing Available Tags

Builds on: Byte 1, Byte 2

In plain terms: This endpoint provides a clean list of all tags currently stored in the system, sorted alphabetically. It gives user interfaces an easy way to populate tag dropdowns and autocomplete menus.

The code:

```
@router.get("/tags", response_model=list[TagResponse])
def get_tags(db: Session = Depends(get_db)):
    return db.query(models.Tag).order_by(models.Tag.name.asc()).all()
```

The decorator `@router.get("/tags")` binds HTTP GET requests sent to `/contacts/tags` to this function.

FastAPI's `Depends(get_db)` injects an open database session that automatically closes when the request finishes. The `response_model=list[TagResponse]` parameter uses a **Pydantic schema** (a data validation and serialization model) to guarantee that only the fields defined in `TagResponse` are serialized into the outgoing JSON response.

Byte 4: Contact Creation & Uniqueness Protection

Builds on: Byte 1, Byte 2

In plain terms: This endpoint accepts new contact information from an incoming request, assigns tags to it, and saves it to the database. If the email or phone number is already registered, it stops the operation and sends back a readable error message.

The code:

```
@router.post("/", response_model=ContactResponse)
def create_contact(
    contact: ContactCreate,
    db: Session = Depends(get_db),
):
    new_contact = models.Contact(
        name=contact.name,
        phone_number=contact.phone_number,
        email=contact.email,
        address=contact.address,
    )

    new_contact.tags = get_or_create_tags(db, contact.tags)
    db.add(new_contact)

    try:
        db.commit()
        db.refresh(new_contact)
    except IntegrityError:
        db.rollback()
        raise HTTPException(
            status_code=400,
            detail="Phone number or email already exists",
        )

    return new_contact
```

The database table has unique constraints on email and phone_number. If a client tries to insert an existing value, the database raises an **IntegrityError**.

Wrapping db.commit() in a try/except block prevents an unhandled database collision from crashing the request with a generic 500 Internal Server Error. Instead, db.rollback() resets the transaction, and HTTPException(status_code=400, ...) informs the client precisely why their request was rejected.

Byte 5: Reading a Single Contact

Builds on: Byte 1

In plain terms: This endpoint looks up a specific contact by their unique numeric ID. If the contact exists, it returns their data; if not, it signals to the caller that the record was not found.

The code:

```
@router.get("/{contact_id}", response_model=ContactResponse)
def get_contact(
    contact_id: int,
    db: Session = Depends(get_db),
):
    contact = (
        db.query(models.Contact)
        .filter(models.Contact.id == contact_id)
        .first()
    )

    if contact is None:
        raise HTTPException(
            status_code=404,
            detail="Contact not found",
        )

    return contact
```

The path parameter {contact_id} is extracted by FastAPI, converted to an integer, and passed directly into the query filter.

If .first() finds no matching record, it returns None. The explicit check raises an HTTP 404 status code, maintaining standard REST conventions.

Byte 6: Partial Updates & Field Patching

Builds on: Byte 2, Byte 4, Byte 5

In plain terms: When editing a contact, users often want to change just one field—like an address—without re-sending the whole profile. This endpoint updates only the specific fields provided by the caller while keeping the rest intact.

The code:

```

@router.put("/{contact_id}", response_model=ContactResponse)
def update_contact(
    contact_id: int,
    contact_data: ContactUpdate,
    db: Session = Depends(get_db),
):
    contact = (
        db.query(models.Contact)
        .filter(models.Contact.id == contact_id)
        .first()
    )

    if contact is None:
        raise HTTPException(
            status_code=404,
            detail="Contact not found",
        )

    update_data = contact_data.model_dump(exclude_unset=True)
    tag_names = update_data.pop("tags", None)

    for field, value in update_data.items():
        setattr(contact, field, value)

    if tag_names is not None:
        contact.tags = get_or_create_tags(db, tag_names)

    try:
        db.commit()
        db.refresh(contact)
    except IntegrityError:
        db.rollback()
        raise HTTPException(
            status_code=400,
            detail="Phone number or email already exists",
        )

    return contact

```

The critical method here is `contact_data.model_dump(exclude_unset=True)`. It converts the incoming Pydantic model into a dictionary containing **only** the keys explicitly supplied by the client, ignoring omitted defaults.

Tags are pulled out separately so they can run through `get_or_create_tags()`, while other fields are assigned dynamically using Python's `setattr()`. The `IntegrityError` block guards against updating a phone number or email to one already held by another contact.

Byte 7: Deleting a Contact

Builds on: Byte 5

In plain terms: This endpoint permanently removes an individual contact record from the database based on their ID.

The code:

```

@router.delete("/{contact_id}")
def delete_contact(
    contact_id: int,
    db: Session = Depends(get_db),
):
    contact = (
        db.query(models.Contact)
        .filter(models.Contact.id == contact_id)
        .first()
    )

    if contact is None:
        raise HTTPException(
            status_code=404,
            detail="Contact not found",
        )

    db.delete(contact)
    db.commit()

    return {"message": "Contact deleted successfully"}

```

Before attempting a deletion, the code queries the database to confirm the target entity actually exists.

If the contact is missing, an immediate 404 is returned. If found, `db.delete(contact)` marks the ORM instance for removal, and `db.commit()` executes the SQL DELETE statement.

Byte 8: Updating Activity Timestamps

Builds on: Byte 5

In plain terms: This endpoint marks the exact time a user viewed or opened a contact record. It acts as an audit trail and enables "recently viewed" features.

The code:

```
@router.patch("/{contact_id}/viewed", response_model=ContactResponse)
def mark_contact_viewed(
    contact_id: int,
    db: Session = Depends(get_db),
):
    contact = (
        db.query(models.Contact)
        .filter(models.Contact.id == contact_id)
        .first()
    )

    if contact is None:
        raise HTTPException(status_code=404, detail="Contact not found")

    contact.last_viewed_at = datetime.utcnow()
    db.commit()
    db.refresh(contact)

    return contact
```

An HTTP **PATCH** verb is used here because this action modifies only a single metadata field (`last_viewed_at`) rather than replacing the resource.

Setting `datetime.utcnow()` writes the current UTC timestamp, and `db.refresh(contact)` reloads the database state into memory so the returned response reflects the updated timestamp.

Byte 9: Dashboard Metrics Calculation

Builds on: Byte 1

In plain terms: This endpoint computes high-level summary counts—such as total contacts, favorites, untagged contacts, and entries added within the past week—for an analytics dashboard.

The code:

```
@router.get("/metrics", response_model=ContactMetricsResponse)
def get_contact_metrics(db: Session = Depends(get_db)):
    total = db.query(func.count(models.Contact.id)).scalar() or 0

    favorites = (
        db.query(func.count(models.Contact.id))
        .filter(models.Contact.is_favorite.is_(True))
        .scalar()
        or 0
    )

    unlabeled = (
        db.query(func.count(models.Contact.id))
        .outerjoin(models.Contact.tags)
        .filter(models.Tag.id.is_(None))
        .scalar()
        or 0
    )

    recently_added = (
        db.query(func.count(models.Contact.id))
        .filter(models.Contact.created_at >= datetime.utcnow() - timedelta(days=7))
        .scalar()
        or 0
    )

    return {
        "total": total,
        "favorites": favorites,
        "recently_added": recently_added,
        "unlabeled": unlabeled,
    }
```

Instead of loading thousands of contact rows into memory to count them with `len()`, this code delegates calculation directly to the SQL engine using `func.count()`.

The `.scalar()` method extracts the single resulting integer value directly. Finding "unlabeled" contacts uses an **outerjoin**: it matches contacts to tags and filters for records where the tag side produces a NULL (`Tag.id.is_(None)`).

Byte 10: Dynamic Search & Multi-Criteria Filtering

Builds on: Byte 1, Byte 2

In plain terms: This block acts as a search engine for contacts, building a custom database query on the fly depending on what search keywords or filters (favorites, specific tags, unlabeled) the user applied.

The code:

```
query = db.query(models.Contact)

if search:
    search_pattern = f"%{search.strip()}%"
    query = query.filter(
        (models.Contact.name.ilike(search_pattern))
        | (models.Contact.phone_number.ilike(search_pattern))
        | exists(
            models.contact_tags.join(models.Tag.__table__)
            .select()
            .where(
                (models.contact_tags.c.contact_id == models.Contact.id)
                & (models.contact_tags.c.tag_id == models.Tag.id)
                & models.Tag.name.ilike(search_pattern)
            )
        )
    )

if favorite is not None:
    query = query.filter(models.Contact.is_favorite == favorite)

if tag:
    query = query.join(models.Contact.tags).filter(
        models.Tag.name.ilike(tag)
    )

if unlabeled:
    query = query.outerjoin(models.Contact.tags).filter(
        models.Tag.id.is_(None)
    )

if recent:
    query = query.filter(models.Contact.last_viewed_at.is_not(None))
```

SQLAlchemy queries are lazy and chainable: calling `.filter()` does not run a query immediately; it builds an abstract syntax tree in memory.

The search parameter searches across contact names, phone numbers, and associated tags via a correlated `exists()` subquery. The `.ilike()` operator provides case-insensitive wildcard matching (e.g., `%alice%` matches "Alice", "ALICE", or "Malice").

Byte 11: Sorting, Offset Pagination & Result Delivery

Builds on: Byte 10

In plain terms: Once contacts are filtered, this code determines their display order and delivers them in small, manageable pages so the server and client aren't overloaded by giant responses.

The code:

```
if sort == "name_desc":
    query = query.order_by(models.Contact.name.desc())
elif sort == "recently_viewed":
    query = query.order_by(
        models.Contact.last_viewed_at.desc().nullslast(),
        models.Contact.name.asc(),
    )
elif sort == "recently_added":
    query = query.order_by(models.Contact.created_at.desc())
elif sort == "recently_updated":
    query = query.order_by(models.Contact.updated_at.desc())
else:
    query = query.order_by(models.Contact.name.asc())

total = query.count()
offset = (page - 1) * limit

contacts = query.offset(offset).limit(limit).all()

return {
    "contacts": contacts,
```

```

        "page": page,
        "limit": limit,
        "total": total,
    }

```

This completes the `get_contacts` function. The query ordering handles edge cases like `nullslast()`—ensuring contacts that have never been viewed appear at the bottom rather than the top when sorting by `recently_viewed`.

Pagination uses the standard SQL `OFFSET` and `LIMIT` formulas: page 1 with limit 10 fetches rows 0–9, page 2 fetches rows 10–19. The total matching count is computed before applying the slice, giving the frontend enough metadata to render page controls.

Byte 12: Bulk Ingestion with Nested Savepoints

Builds on: Byte 2, Byte 4

In plain terms: When importing a spreadsheet or batch file with hundreds of contacts, a single duplicate row shouldn't abort the entire upload. This endpoint loops through incoming records, saving valid contacts while logging failures for invalid ones.

The code:

```

@router.post("/import", response_model=ContactImportResponse)
def import_contacts(
    payload: ContactImportRequest,
    db: Session = Depends(get_db),
):
    imported = 0
    skipped = 0
    errors = []

    for row_number, contact in enumerate(payload.rows, start=2):
        try:
            with db.begin_nested():
                new_contact = models.Contact(
                    name=contact.name,
                    phone_number=contact.phone_number,
                    email=contact.email,
                    address=contact.address,
                )

                new_contact.tags = get_or_create_tags(db, contact.tags)
                db.add(new_contact)
                db.flush()

                imported += 1

        except IntegrityError:
            skipped += 1
            errors.append({
                "row": row_number,
                "reason": "Duplicate phone number or email",
            })

    db.commit()

    return {
        "imported": imported,
        "skipped": skipped,
        "errors": errors,
    }

```

In standard database transactions, encountering an `IntegrityError` breaks the whole transaction, preventing subsequent inserts.

The `with db.begin_nested():` context manager establishes a database **savepoint** for each row. If row 5 triggers a unique constraint collision on an email, only row 5 rolls back. The transaction stays healthy, allowing rows 1–4 and 6 onward to persist when `db.commit()` is finally executed. `row_number` starts at 2 to mirror conventional spreadsheet row numbering (accounting for header rows).

User prompt: import os from dotenv import load_dotenv from sqlalchemy import create_engine from sqlalchemy.orm import declarative_base, sessionmaker load_dotenv() DATABASE_URL = (f"postgresql+psycopg://\" f"{os.getenv('DATABASE_USER')}:\" f\"{os.getenv('DATABASE_PASSWORD')}\"@\" f\"{os.getenv('DATABASE_HOST')}:\" f\"{os.getenv('DATABASE_PORT')}\"/\" f\"{os.getenv('DATABASE_NAME')}\") engine = create_engine(DATABASE_URL) SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine) Base = declarative_base() def get_db(): db = SessionLocal() try: yield db finally: db.close()

Response: Byte 1: Environment Variables & Database URL Assembly

Builds on: None — starting point

In plain terms: Before an application can talk to a database, it needs connection credentials like the server address, username, and password. Instead of hardcoding sensitive passwords into source files, this code reads them from environment variables and constructs a standard database connection address (URI).

The code:

```
import os

from dotenv import load_dotenv

load_dotenv()

DATABASE_URL = (
    f"postgresql+psycopg://{os.getenv('DATABASE_USER')}:"
    f"{os.getenv('DATABASE_PASSWORD')}}@{os.getenv('DATABASE_HOST')}:"
    f"{os.getenv('DATABASE_PORT')}/{os.getenv('DATABASE_NAME')}"
)
```

The function `load_dotenv()` loads key-value pairs from a local `.env` file into system environment variables so `os.getenv(...)` can read them.

The resulting `DATABASE_URL` string follows the standard database URI scheme: `driver://user:password@host:port/dbname`. The dialect `postgresql+psycopg` instructs SQLAlchemy to communicate with PostgreSQL using modern `psycopg (v3)` as the underlying database driver.

Byte 2: The Database Engine

Builds on: Byte 1

In plain terms: The engine is the central foundation of database connectivity. It manages the low-level communication line, runs raw queries, and maintains a pool of reusable connections to avoid the heavy overhead of reconnecting on every request.

The code:

```
from sqlalchemy import create_engine

engine = create_engine(DATABASE_URL)

create_engine() sets up an interface between Python and PostgreSQL.
```

It does not open a database connection immediately upon declaration; instead, it establishes a connection pool that opens and hands out actual database connections lazily when queries or sessions demand them.

Byte 3: Session Factory (SessionLocal)

Builds on: Byte 2

In plain terms: While the engine handles low-level pipes, a "session" is your workspace where you track, add, and query Python objects before committing them to tables. This block configures a reusable template—a factory—that stamps out consistent database sessions.

The code:

```
from sqlalchemy.orm import sessionmaker

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)
```

`sessionmaker` creates a callable class (`SessionLocal`). Every time you call `SessionLocal()`, it instantiates a distinct SQLAlchemy Session bound to the engine.

Setting `autocommit=False` ensures changes aren't instantly saved; you must explicitly call `db.commit()` to finalize transactions. Setting `autoflush=False` prevents SQLAlchemy from automatically sending uncommitted changes to the database before every query, giving the developer explicit control over when queries synchronize with the database.

Byte 4: Declarative Base for ORM Models

Builds on: Byte 2

In plain terms: To map database tables to Python classes (like a `Contact` or `Tag` model), those classes need a common ancestor that knows how to register tables with SQLAlchemy. This line creates that base parent class.

The code:

```
from sqlalchemy.orm import declarative_base

Base = declarative_base()

declarative_base() produces a base class that all ORM model classes in your project inherit from (e.g., class Contact(Base): ...).
```

When models inherit from `Base`, SQLAlchemy tracks their field types, relationships, and table metadata, enabling tools to automatically create schema tables via `Base.metadata.create_all(bind=engine)`.

Byte 5: Per-Request Session Lifecycle (get_db)

Builds on: Byte 3

In plain terms: Web requests need their own isolated database session that opens when a request arrives and reliably closes when it ends—even if the request crashes with an error. This generator function manages that exact lifecycle.

The code:

```
def get_db():
    db = SessionLocal()

    try:
        yield db
    finally:
        db.close()
```

This function is a Python generator designed to work with FastAPI's dependency injection system (`db: Session = Depends(get_db)`).

When a route starts, `SessionLocal()` opens a session and `yield` hands it to the route handler. When the route finishes processing (or raises an uncaught exception), execution resumes in the `finally` block, guaranteeing that `db.close()` runs and releases the connection back to the connection pool to prevent connection leaks.

User prompt: from fastapi import FastAPI from fastapi.middleware.cors import CORSMiddleware from fastapi import Depends, HTTPException from sqlalchemy.orm import Session from .database import Base, engine from . import models from .routes.contacts import router as contacts_router from . import models, schemas from .database import get_db app = FastAPI(title="Phonebook API", description="REST API for managing contacts", version="1.0.0") # Create database tables Base.metadata.create_all(bind=engine) @app.get("/") def root(): return { "message": "Phonebook API is running" } # Register contact routes app.include_router(contacts_router) @app.patch("/{contact_id}/favorite") def update_favorite(contact_id: int, favorite: schemas.FavoriteUpdate, db: Session = Depends(get_db),): contact = (db.query(models.Contact) .filter(models.Contact.id == contact_id) .first()) if not contact: raise HTTPException(status_code=404, detail="Contact not found",) contact.is_favorite = favorite.is_favorite db.commit() db.refresh(contact) return contact

Response: Byte 1: FastAPI Application Initialization & Metadata

Builds on: None — starting point

In plain terms: This is the root entry point of the entire web application. It creates the core web server instance and configures descriptive details like title and version for documentation.

The code:

```
app = FastAPI(
    title="Phonebook API",
    description="REST API for managing contacts",
    version="1.0.0"
)
```

The `FastAPI` class instance (`app`) is the ASGI application object that web servers (like Uvicorn) run.

Providing metadata like `title`, `description`, and `version` automatically populates the interactive API documentation pages generated at `/docs` (Swagger UI) and `/redoc` (ReDoc).

Byte 2: Automatic Database Table Creation

Builds on: Byte 1, previous `database.py` (Bytes 2 & 4)

In plain terms: When the application server starts up, this line checks the database to verify whether all tables defined in your Python models exist. Any tables that are missing are created automatically.

The code:

```
# Create database tables
Base.metadata.create_all(bind=engine)
```

`Base.metadata` collects metadata about all SQLAlchemy classes inheriting from `Base` (such as `Contact` and `Tag`).

When `.create_all(bind=engine)` runs on startup, it emits `CREATE TABLE IF NOT EXISTS SQL` commands across the database connection pool. While convenient for prototypes and tests, production setups typically swap this out for migration engines like Alembic to manage structural changes over time.

Byte 3: Health Check / Root Endpoint

Builds on: Byte 1

In plain terms: This endpoint provides a simple greeting message at the base URL (`/`). It serves as an immediate check to confirm the server is online and responding.

The code:

```
@app.get("/")
def root():
```

```

    return {
        "message": "Phonebook API is running"
    }

```

A root route is widely used as a **health check** or **liveness probe** by container orchestrators (like Docker or Kubernetes) and load balancers.

FastAPI automatically serializes the returned Python dictionary into a JSON payload with a 200 OK status code.

Byte 4: Modular Router Mounting

Builds on: Byte 1, contacts router (Byte 1)

In plain terms: Instead of cluttering the main app file with dozens of endpoints, this line connects the modular contacts router created in another file directly into the application.

The code:

```

# Register contact routes
app.include_router(contacts_router)

```

`include_router()` attaches all routes defined on `contacts_router` (such as `GET /contacts/`, `POST /contacts/`, `DELETE /contacts/{id}`) into the parent application.

This keeps the codebase cleanly separated into modules by domain responsibility while maintaining a single cohesive API.

Byte 5: Toggling Favorite Status (Targeted PATCH)

Builds on: Byte 1, contacts router (Byte 5 & Byte 8)

In plain terms: This endpoint updates whether a contact is starred or marked as a favorite. Because it modifies only a single attribute of a contact, it uses an HTTP PATCH method.

The code:

```

@app.patch("/contacts/{contact_id}/favorite")
def update_favorite(
    contact_id: int,
    favorite: schemas.FavoriteUpdate,
    db: Session = Depends(get_db),
):
    contact = (
        db.query(models.Contact)
        .filter(models.Contact.id == contact_id)
        .first()
    )

    if not contact:
        raise HTTPException(
            status_code=404,
            detail="Contact not found",
        )

    contact.is_favorite = favorite.is_favorite

    db.commit()
    db.refresh(contact)

    return contact

```

The handler expects a lightweight payload validated by `schemas.FavoriteUpdate` (e.g., `{"is_favorite": true}`).

It looks up the contact record, raises an HTTP 404 if missing, updates the boolean flag `contact.is_favorite`, commits the change to the database, and returns the refreshed contact model.

Byte 6: Route Placement & Unused Imports (Gotcha / Code Smell)

Builds on: Byte 4, Byte 5

In plain terms: This file contains minor structural inconsistencies: route definitions that belong inside a sub-module are sitting in the main file, and several tools were imported into memory but never actually used.

The code:

```

from fastapi.middleware.cors import CORSMiddleware
from . import models, schemas

# Defined on app directly instead of inside contacts_router:
@app.patch("/contacts/{contact_id}/favorite")
def update_favorite(...):
    ...

```

Notice two gotchas here:

1. **Unused Imports & Middleware:** `CORSMiddleware` is imported but never attached via `app.add_middleware()`, meaning cross-origin frontend apps (like a React or Vue dashboard on another port) might fail to make requests due to CORS restrictions. Additionally, `models` is imported twice on lines 6 and 8.
2. **Scattered Domain Logic:** The endpoint `@app.patch("/contacts/{contact_id}/favorite")` manages contact domain logic, but lives in `main.py` instead of inside `routes/contacts.py` alongside the rest of the `/contacts` endpoints. Moving this route inside `contacts_router` would keep the API structure consistent and self-contained.

User prompt: from sqlalchemy import Boolean, Column, DateTime, ForeignKey, Integer, String, Table, Text from sqlalchemy.orm import relationship from .database import Base from datetime import datetime contact_tags = Table("contact_tags", Base.metadata, Column("contact_id", ForeignKey("contacts.id", ondelete="CASCADE"), primary_key=True), Column("tag_id", ForeignKey("tags.id", ondelete="CASCADE"), primary_key=True),) class Contact(Base): __tablename__ = "contacts" id = Column(Integer, primary_key=True, index=True) name = Column(String(255), nullable=False) phone_number = Column(String(50), unique=True, nullable=False) email = Column(String(255), unique=True, nullable=True) address = Column(Text, nullable=True) is_favorite = Column(Boolean, default=False, nullable=False) last_viewed_at = Column(DateTime, nullable=True) created_at = Column(DateTime, default=datetime.utcnow, nullable=False) updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow, nullable=False) tags = relationship("Tag", secondary=contact_tags, back_populates="contacts", lazy="joined",) class Tag(Base): __tablename__ = "tags" id = Column(Integer, primary_key=True, index=True) name = Column(String(80), unique=True, nullable=False, index=True) contacts = relationship("Contact", secondary=contact_tags, back_populates="tags",)

Response: Byte 1: The Association Table (contact_tags)

Builds on: Previous `database.py` (Byte 4)

In plain terms: A single contact can have many tags (like "Family" and "Work"), and a single tag can belong to many contacts. To represent this many-to-many relationship in a relational database, you need a middleman table whose only job is linking pairs of IDs together.

The code:

```
contact_tags = Table(
    "contact_tags",
    Base.metadata,
    Column("contact_id", ForeignKey("contacts.id", ondelete="CASCADE"), primary_key=True),
    Column("tag_id", ForeignKey("tags.id", ondelete="CASCADE"), primary_key=True),
)
```

SQLAlchemy uses a core `Table` object (rather than a full ORM class) when the table stores pure associations without extra data columns.

Setting both `contact_id` and `tag_id` as `primary_key=True` creates a **composite primary key**, ensuring the exact same tag cannot be attached to the same contact twice. The `ondelete="CASCADE"` constraint tells the database engine that if a contact or tag is deleted, any matching junction rows here should be removed automatically.

Byte 2: Core Contact Identity & Contact Details

Builds on: None — starting point

In plain terms: This part of the `Contact` model defines the fundamental real-world data columns for a person, including their name, phone number, email, and physical address.

The code:

```
class Contact(Base):
    __tablename__ = "contacts"

    id = Column(Integer, primary_key=True, index=True)

    name = Column(String(255), nullable=False)

    phone_number = Column(
        String(50),
        unique=True,
        nullable=False
    )

    email = Column(
        String(255),
        unique=True,
        nullable=True
    )

    address = Column(
        Text,
        nullable=True
    )
```

Inheriting from `Base` registers the class as a database table named "contacts".

Notice the constraints:

- `name` and `phone_number` have `nullable=False`, making them mandatory.

- Both `phone_number` and `email` enforce `unique=True` at the database level. This is the exact constraint that triggers the `IntegrityError` handled in the API endpoints.
- `address` uses `Text` (unlimited length) rather than `String(255)` to accommodate long or multi-line mailing addresses.

Byte 3: Status & Automatic Audit Timestamps

Builds on: Byte 2

In plain terms: These columns track user interactions and lifecycle events for each contact—flagging favorites, noting when a contact was last viewed, and recording when the entry was created or edited.

The code:

```
is_favorite = Column(
    Boolean,
    default=False,
    nullable=False
)

last_viewed_at = Column(
    DateTime,
    nullable=True
)

created_at = Column(
    DateTime,
    default=datetime.utcnow,
    nullable=False
)

updated_at = Column(
    DateTime,
    default=datetime.utcnow,
    onupdate=datetime.utcnow,
    nullable=False
)
```

The `is_favorite` flag defaults to `False` whenever a contact is created without one specified.

For `created_at` and `updated_at`, notice that `datetime.utcnow` is passed as a **function reference** (callable without parentheses `()`). This ensures the timestamp evaluates at the moment of insertion or update rather than when the application starts. The `onupdate` parameter automatically overwrites `updated_at` with a fresh timestamp whenever any column in that row changes.

Byte 4: The Tag Model & Bi-Directional Relationship

Builds on: Byte 1, Byte 2

In plain terms: This class defines the `Tag` table and establishes a two-way bridge between contacts and tags. It lets you access tags directly from a contact (`contact.tags`) and contacts directly from a tag (`tag.contacts`).

The code:

```
class Tag(Base):
    __tablename__ = "tags"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(80), unique=True, nullable=False, index=True)

    contacts = relationship(
        "Contact",
        secondary=contact_tags,
        back_populates="tags",
    )
```

(and on `Contact`):

```
tags = relationship(
    "Tag",
    secondary=contact_tags,
    back_populates="contacts",
    lazy="joined",
)
```

The `relationship()` construct is a high-level SQLAlchemy abstraction; it does not create a physical column in the database table.

Setting `secondary=contact_tags` informs SQLAlchemy to navigate through the junction table defined in Byte 1. `back_populates` links both sides together so adding a tag to `contact.tags` immediately updates `tag.contacts` in memory.

Byte 5: Eager Loading Strategy (`lazy="joined"`)

Builds on: Byte 4

In plain terms: By default, databases don't load related data until you specifically ask for it, which can cause dozens of extra queries behind the scenes. This configuration ensures that whenever a contact is fetched, its associated tags are pulled along in the exact same database query.

The code:

```
tags = relationship(
    "Tag",
    secondary=contact_tags,
    back_populates="contacts",
    lazy="joined",
)
```

In standard SQLAlchemy relationships (`lazy="select"`), accessing `.tags` on ten contacts triggers one query for the contacts plus ten secondary queries to fetch each contact's tags—a classic performance bottleneck known as the **N+1 query problem**.

Configuring `lazy="joined"` instructs SQLAlchemy to execute a SQL `LEFT OUTER JOIN` immediately whenever `Contact` is loaded, retrieving both the contact and all associated tags in a single round-trip to the database.

User prompt: from datetime import datetime import re from pydantic import BaseModel, ConfigDict, EmailStr, Field, field_validator class ContactBase(BaseModel): model_config = ConfigDict(extra="forbid") name: str = Field(..., max_length=255) phone_number: str = Field(..., max_length=20) email: EmailStr | None = None address: str | None = None tags: list[str] = Field(default_factory=list, max_length=10) @field_validator("phone_number") @classmethod def validate_phone_number(cls, value: str) -> str: value = value.strip() digits = "".join(character for character in value if character.isdigit()) if (not value or len(digits) < 7 or len(digits) > 15 or not re.fullmatch(r"^[0-9\s()|-]+", value)): raise ValueError("Enter a valid phone number with 7–15 digits.") return value @field_validator("tags") @classmethod def validate_tags(cls, value: list[str]) -> list[str]: cleaned = [] seen = set() for tag in value: name = str(tag).strip() key = name.lower() if not name or key in seen: continue if len(name) > 80: raise ValueError("Each tag must be 80 characters or fewer.") seen.add(key) cleaned.append(name) if len(cleaned) > 10: raise ValueError("A contact can have at most 10 tags.") return cleaned class ContactCreate(ContactBase): pass class ContactUpdate(ContactBase): model_config = ConfigDict(extra="forbid") name: str | None = Field(None, max_length=255) phone_number: str | None = Field(None, max_length=20) email: EmailStr | None = None address: str | None = None tags: list[str] | None = Field(None, max_length=10) @field_validator("phone_number") @classmethod def validate_phone_number(cls, value: str | None) -> str | None: if value is None: return value value = value.strip() digits = "".join(character for character in value if character.isdigit()) if (not value or len(digits) < 7 or len(digits) > 15 or not re.fullmatch(r"^[0-9\s()|-]+", value)): raise ValueError("Enter a valid phone number with 7–15 digits.") return value @field_validator("tags") @classmethod def validate_tags(cls, value: list[str] | None) -> list[str] | None: if value is None: return value cleaned = [] seen = set() for tag in value: name = str(tag).strip() key = name.lower() if not name or key in seen: continue if len(name) > 80: raise ValueError("Each tag must be 80 characters or fewer.") seen.add(key) cleaned.append(name) if len(cleaned) > 10: raise ValueError("A contact can have at most 10 tags.") return cleaned class TagResponse(BaseModel): id: int name: str model_config = ConfigDict(from_attributes=True) class ContactResponse(BaseModel): id: int name: str phone_number: str email: str | None address: str | None is_favorite: bool last_viewed_at: datetime | None created_at: datetime updated_at: datetime tags: list[TagResponse] = Field(default_factory=list) model_config = ConfigDict(from_attributes=True) class ContactListResponse(BaseModel): contacts: list[ContactResponse] page: int limit: int total: int class ContactMetricsResponse(BaseModel): total: int favorites: int recently_added: int unlabeled: int class ContactImportRequest(BaseModel): rows: list[ContactCreate] = Field(..., max_length=5000) class ContactImportError(BaseModel): row: int reason: str class ContactImportResponse(BaseModel): imported: int skipped: int errors: list[ContactImportError] = Field(default_factory=list) class FavoriteUpdate(BaseModel): is_favorite: bool

Response: Byte 1: Pydantic Base Schemas & Strict Field Validation

Builds on: None — starting point

In plain terms: Before data reaches database queries or endpoints, it must be checked for correct types, lengths, and shapes. This base schema acts as the primary gatekeeper for contact information, rejecting unexpected keys and ensuring required fields are supplied.

The code:

```
class ContactBase(BaseModel):
    model_config = ConfigDict(extra="forbid")

    name: str = Field(..., max_length=255)
    phone_number: str = Field(..., max_length=20)
    email: EmailStr | None = None
    address: str | None = None
    tags: list[str] = Field(default_factory=list, max_length=10)
```

`ConfigDict(extra="forbid")` enforces strict input integrity: if a client attempts to send extra or unmapped fields (like typos or malicious payload attributes), Pydantic immediately rejects the request with a validation error.

The `Field(..., ...)` syntax marks `name` and `phone_number` as required parameters (indicated by the ellipsis `...`), while `EmailStr` automatically verifies standard email syntax rules.

Byte 2: Custom Phone Number & Tag Sanitization

Builds on: Byte 1, previous `contacts.py` (Byte 2)

In plain terms: Standard type annotations cannot tell if a string is a legitimate phone number or whether a list of labels has duplicates. These custom validator functions clean, format, and enforce domain rules on phone numbers and tag collections.

The code:

```
@field_validator("phone_number")
@classmethod
```

```

def validate_phone_number(cls, value: str) -> str:
    value = value.strip()
    digits = "".join(character for character in value if character.isdigit())

    if (
        not value
        or len(digits) < 7
        or len(digits) > 15
        or not re.fullmatch(r"\+?[0-9\s()\-]+", value)
    ):
        raise ValueError("Enter a valid phone number with 7–15 digits.")

    return value

@field_validator("tags")
@classmethod
def validate_tags(cls, value: list[str]) -> list[str]:
    cleaned = []
    seen = set()

    for tag in value:
        name = str(tag).strip()
        key = name.lower()

        if not name or key in seen:
            continue

        if len(name) > 80:
            raise ValueError("Each tag must be 80 characters or fewer.")

        seen.add(key)
        cleaned.append(name)

    if len(cleaned) > 10:
        raise ValueError("A contact can have at most 10 tags.")

    return cleaned

```

@field_validator runs after initial type parsing.

The phone number validator strips outer whitespace, counts raw digits (enforcing the international standard of 7 to 15 digits), and applies a regex to permit common formatting characters like +, (), and -. The tag validator strips whitespace, enforces case-insensitive deduplication using a seen set, caps individual tag length at 80 characters, and limits total tags to 10.

Byte 3: Create vs. Partial Update Schemas

Builds on: Byte 1, Byte 2, previous `contacts.py` (Byte 4 & Byte 6)

In plain terms: Creating a contact requires core details up front, but updating an existing contact should allow sending only the specific fields being edited. This distinction is handled by creating two distinct models: one strict, and one where every field is optional.

The code:

```

class ContactCreate(ContactBase):
    pass

class ContactUpdate(BaseModel):
    model_config = ConfigDict(extra="forbid")

    name: str | None = Field(None, max_length=255)
    phone_number: str | None = Field(None, max_length=20)
    email: EmailStr | None = None
    address: str | None = None
    tags: list[str] | None = Field(None, max_length=10)
    ...

```

ContactCreate simply inherits from ContactBase, keeping name and phone_number mandatory.

ContactUpdate re-declares every field with a default of None. In route handlers, calling `contact_data.model_dump(exclude_unset=True)` distinguishes between a field that was deliberately omitted by the client versus a field explicitly passed as None or updated to a new value.

Byte 4: ORM Deserialization with `from_attributes`

Builds on: Previous `models.py` (Byte 2, Byte 4 & Byte 5)

In plain terms: By default, Pydantic expects data as Python dictionaries (`data["name"]`), but SQLAlchemy outputs database models whose values are accessed as object attributes (`contact.name`). This configuration tells Pydantic how to read directly from database objects.

The code:

```

class TagResponse(BaseModel):
    id: int
    name: str
    model_config = ConfigDict(from_attributes=True)

class ContactResponse(BaseModel):
    id: int
    name: str
    phone_number: str
    email: str | None
    address: str | None
    is_favorite: bool
    last_viewed_at: datetime | None
    created_at: datetime
    updated_at: datetime
    tags: list[TagResponse] = Field(default_factory=list)
    model_config = ConfigDict(from_attributes=True)

```

In Pydantic v1, this behavior was called `orm_mode = True`; in Pydantic v2, it is `model_config = ConfigDict(from_attributes=True)`.

When an endpoint returns a SQLAlchemy models.Contact instance, Pydantic reads attributes like `contact.id` and recursively converts nested relations like `contact.tags` into matching TagResponse objects, stripping any internal database columns not declared in the response model.

Byte 5: Envelope Payloads for Lists and Metrics

Builds on: Byte 4, previous `contacts.py` (Byte 9 & Byte 11)

In plain terms: Instead of returning a bare JSON array of contacts, APIs wrap responses in structured metadata containers. These models define the envelope for paginated contact lists and dashboard analytical counters.

The code:

```

class ContactListResponse(BaseModel):
    contacts: list[ContactResponse]
    page: int
    limit: int
    total: int

class ContactMetricsResponse(BaseModel):
    total: int
    favorites: int
    recently_added: int
    unlabeled: int

```

ContactListResponse packages pagination metadata (page, limit, total) alongside the list of items (contacts). This provides client applications with the necessary information to compute total page counts and render navigation buttons.

ContactMetricsResponse enforces precise typing for all summary counters computed by the `/contacts/metrics` endpoint.

Byte 6: Bulk Ingestion Contracts & Isolated Actions

Builds on: Byte 3, Byte 4, previous `contacts.py` (Byte 12), previous `main.py` (Byte 5)

In plain terms: These schemas define the input and output structures for specialized operations: uploading contacts in bulk and toggling a contact's favorite status.

The code:

```

class ContactImportRequest(BaseModel):
    rows: list[ContactCreate] = Field(..., max_length=5000)

class ContactImportError(BaseModel):
    row: int
    reason: str

class ContactImportResponse(BaseModel):
    imported: int
    skipped: int
    errors: list[ContactImportError] = Field(default_factory=list)

class FavoriteUpdate(BaseModel):
    is_favorite: bool

```

ContactImportRequest caps bulk ingestion at 5,000 rows to prevent memory exhaustion and request timeouts. Each item inside rows must satisfy ContactCreate, meaning individual row data is validated before database insertion occurs.

ContactImportResponse provides detailed feedback on how many rows succeeded, how many were skipped, and an array of ContactImportError objects linking errors to their exact row numbers. Finally, FavoriteUpdate cleanly encapsulates the payload required for the favorite toggle endpoint.

User prompt: `import pytest from fastapi.testclient import TestClient from sqlalchemy import create_engine from sqlalchemy.pool import StaticPool from sqlalchemy.orm import sessionmaker from app.database import Base, get_db from app.main import app @pytest.fixture() def client(): engine = create_engine("sqlite://", connect_args={"check_same_thread": False}, poolclass=StaticPool,) TestingSession = sessionmaker(bind=engine, autocommit=False, autoflush=False) Base.metadata.create_all(bind=engine) def override_get_db(): db = TestingSession() try: yield db finally: db.close() app.dependency_overrides[get_db] = override_get_db with TestClient(app) as test_client: yield test_client app.dependency_overrides.clear() Base.metadata.drop_all(bind=engine) def test_contact_crud_and_duplicate_phone(client): payload = { "name": "Ada Lovelace", "phone_number": "+1 555 0100", "email": "ada@example.com", "tags": ["Work"], } created = client.post("/contacts/", json=payload) assert created.status_code == 200 contact_id = created.json()["id"] assert created.json()["tags"][0] ["name"] == "Work" duplicate = client.post("/contacts/", json={**payload, "email": "other@example.com"}) assert duplicate.status_code == 400 listed = client.get("/contacts/", params={"search": "Ada", "tag": "Work"}) assert listed.status_code == 200 assert listed.json()["total"] == 1 tag_search = client.get("/contacts/", params={"search": "work"}) assert tag_search.status_code == 200 assert tag_search.json()["total"] == 1 combined_search = client.get("/contacts/", params={"search": "Ada", "tag": "Friends"}) assert combined_search.status_code == 200 assert combined_search.json()["total"] == 0 metrics = client.get("/contacts/metrics") assert metrics.status_code == 200 assert metrics.json()["total"] == 1 assert metrics.json()["unlabeled"] == 0 viewed_list = client.get("/contacts/", params={"recent": True, "sort": "recently_viewed"}) assert viewed_list.status_code == 200 assert viewed_list.json()["total"] == 0 updated = client.put(f"/contacts/{contact_id}", json={"name": "Ada Byron", "tags": ["Friends"]}) assert updated.status_code == 200 assert updated.json()["tags"][0]["name"] == "Friends" viewed = client.patch(f"/contacts/{contact_id}/viewed") assert viewed.status_code == 200 assert viewed.json()["last_viewed_at"] is not None deleted = client.delete(f"/contacts/{contact_id}") assert deleted.status_code == 200 assert client.get(f"/contacts/{contact_id}").status_code == 404 def test_validation_requires_name_and_phone(client): response = client.post("/contacts/", json={"name": "Missing phone"}) assert response.status_code == 422 def test_import_validates_rows_and_skips_duplicates(client): response = client.post("/contacts/import", json={ "rows": [{"name": "Imported One", "phone_number": "90000000001", "email": "one@example.com", "address": None, "tags": []}, {"name": "Duplicate", "phone_number": "90000000001", "email": "two@example.com", "address": None, "tags": []},], }) assert response.status_code == 200 assert response.json()["imported"] == 1 assert response.json()["skipped"] == 1 assert response.json()["errors"][0]["row"] == 3 forbidden = client.post("/contacts/import", json={"rows": [{"name": "Unsafe", "phone_number": "90000000002", "id": 99}], }) assert forbidden.status_code == 422`

Response: Byte 1: The Isolated In-Memory Database Fixture

Builds on: Previous database.py (Bytes 2, 4 & 5), main.py (Byte 1)

In plain terms: Automated tests should never write to or wipe out your real database. This fixture spins up an ephemeral, in-memory SQLite database solely for the test run and discards it completely when tests finish.

The code:

```
@pytest.fixture()
def client():
    engine = create_engine(
        "sqlite://",
        connect_args={"check_same_thread": False},
        poolclass=StaticPool,
    )
    TestingSession = sessionmaker(bind=engine, autocommit=False, autoflush=False)
    Base.metadata.create_all(bind=engine)
    ...
    Base.metadata.drop_all(bind=engine)
```

By supplying the URI "sqlite://", SQLite creates the database in RAM with zero disk footprints.

Because in-memory SQLite instances are scoped to single connections by default, poolclass=StaticPool forces all threads and operations to share that exact same memory space. connect_args={"check_same_thread": False} allows FastAPI background tasks or worker threads to access the connection safely. Base.metadata.create_all generates fresh schema tables, while drop_all guarantees clean teardown.

Byte 2: FastAPI Dependency Overriding & TestClient

Builds on: Byte 1, previous database.py (Byte 5)

In plain terms: The application code defaults to connecting to PostgreSQL via get_db. This block intercepts that mechanism during tests, swapping the real database connection for our in-memory SQLite session without modifying any production application files.

The code:

```
def override_get_db():
    db = TestingSession()
    try:
        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = override_get_db
with TestClient(app) as test_client:
```

```

    yield test_client
    app.dependency_overrides.clear()

```

FastAPI provides an explicit hook named `app.dependency_overrides`. By assigning `app.dependency_overrides[get_db] = override_get_db`, any endpoint using `Depends(get_db)` receives `TestingSession` instead of the production `SessionLocal`.

FastAPI's `TestClient` wraps the application using the `httpx` engine, letting you send synthetic HTTP requests directly into the FastAPI router without starting an external web server process. After testing, `dependency_overrides.clear()` restores the app's default state.

Byte 3: Testing the Full Contact Lifecycle (Integration Test)

Builds on: Byte 2, previous `contacts.py` (Bytes 4, 6, 7 & 8)

In plain terms: Instead of testing operations in isolation, this workflow simulates a real user journey: creating a contact, updating their tags, marking them as viewed, deleting them, and verifying that fetching them afterward produces an expected 404 Not Found.

The code:

```

# 1. Create
created = client.post("/contacts/", json=payload)
assert created.status_code == 200
contact_id = created.json()["id"]
assert created.json()["tags"][0]["name"] == "Work"
...
# 2. Update
updated = client.put(f"/contacts/{contact_id}", json={"name": "Ada Byron", "tags": ["Friends"]})
assert updated.status_code == 200
assert updated.json()["tags"][0]["name"] == "Friends"

# 3. Mark Viewed
viewed = client.patch(f"/contacts/{contact_id}/viewed")
assert viewed.status_code == 200
assert viewed.json()["last_viewed_at"] is not None

# 4. Delete & Verify Missing
deleted = client.delete(f"/contacts/{contact_id}")
assert deleted.status_code == 200
assert client.get(f"/contacts/{contact_id}").status_code == 404

```

This sequence tests the complete CRUD (Create, Read, Update, Delete) cycle across state changes.

Notice how state flows dynamically: `contact_id` is extracted from the POST response payload and interpolated into subsequent PUT, PATCH, and DELETE path parameters. The final assertion asserts that requesting an already-deleted resource properly triggers an HTTP 404.

Byte 4: Testing Constraints, Search Queries, and Metrics

Builds on: Byte 3, previous `contacts.py` (Bytes 9, 10 & 11)

In plain terms: This section checks the system's business rules and querying capabilities: verifying that duplicate phone numbers are blocked with a 400 status, tag/name queries return expected records, and metric counters increment accurately.

The code:

```

duplicate = client.post("/contacts/", json={**payload, "email": "other@example.com"})
assert duplicate.status_code == 400

listed = client.get("/contacts/", params={"search": "Ada", "tag": "Work"})
assert listed.status_code == 200
assert listed.json()["total"] == 1

tag_search = client.get("/contacts/", params={"search": "work"})
assert tag_search.status_code == 200
assert tag_search.json()["total"] == 1

combined_search = client.get("/contacts/", params={"search": "Ada", "tag": "Friends"})
assert combined_search.status_code == 200
assert combined_search.json()["total"] == 0

metrics = client.get("/contacts/metrics")
assert metrics.status_code == 200
assert metrics.json()["total"] == 1
assert metrics.json()["unlabeled"] == 0

```

`client.get(..., params={...})` serializes query string parameters (e.g., `?search=Ada&tag=Work`).

Notice the edge cases verified here:

- **Duplicate Detection:** Attempting to reuse Ada's existing phone number with a distinct email triggers an `IntegrityError` resulting in an HTTP 400.
- **Correlated Tag Search:** Searching for the string "work" matches because the search query inspects tag names via the junction table subquery.

- **Filter Intersections:** Searching for Ada with an mismatched tag (Friends) confirms that filters combine via logical AND.

Byte 5: Testing Input Schema Validation & Forbidden Fields

Builds on: Byte 2, previous `schemas.py` (Bytes 1 & 2)

In plain terms: These tests verify that malformed payloads never reach the database. If an API consumer omits required fields or injects unauthorized parameters, the server rejects the request immediately at the validation boundary.

The code:

```
def test_validation_requires_name_and_phone(client):
    response = client.post("/contacts/", json={"name": "Missing phone"})
    assert response.status_code == 422

(and inside test_import_validates_rows_and_skips_duplicates:)

    forbidden = client.post(
        "/contacts/import",
        json={"rows": [{"name": "Unsafe", "phone_number": "90000000002", "id": 99}]},
    )
    assert forbidden.status_code == 422
```

FastAPI automatically responds with HTTP status **422 Unprocessable Entity** when an incoming payload violates a Pydantic schema contract.

In the first test, omitting `phone_number` triggers a 422. In the second test, injecting an unexpected `"id": 99` field violates `model_config = ConfigDict(extra="forbid")` configured on `ContactBase`, proving that client-supplied IDs cannot bypass the schema.

Byte 6: Testing Resilient Batch Ingestion & Row Tracking

Builds on: Byte 2, previous `contacts.py` (Byte 12)

In plain terms: When importing records in bulk, valid items should save while duplicates are skipped, and the server should report the exact row number where any conflict occurred. This test validates that fault-tolerant behavior.

The code:

```
def test_import_validates_rows_and_skips_duplicates(client):
    response = client.post(
        "/contacts/import",
        json={
            "rows": [
                {"name": "Imported One", "phone_number": "90000000001", "email": "one@example.com", "address": "123 Main St"},
                {"name": "Duplicate", "phone_number": "90000000001", "email": "two@example.com", "address": "456 Main St"}
            ]
        },
    )

    assert response.status_code == 200
    assert response.json()["imported"] == 1
    assert response.json()["skipped"] == 1
    assert response.json()["errors"][0]["row"] == 3
```

The payload contains two rows sharing the identical phone number 90000000001.

Because row 1 commits within its nested transaction savepoint, row 2 collides on the database unique constraint and triggers a rollback of that specific row. The assertions verify that the endpoint remains partially successful (`imported: 1`, `skipped: 1`) and identifies the offending record as `row: 3` (matching the 1-based indexing of a spreadsheet with a header row).